Tigrinho Simulator: um jogo em Python com Programação Orientada a Objetos para criticar o vício em apostas

Davi Menezes dos Santos, Renato Alexandre Moura Pinheiro, Yan Martelleto de Mello, Arthur Lobo Nunes Verçosa
Programação Orientada a Objetos
Universidade Federal de Minas Gerais
Belo Horizonte, Brasil

***Resumo*—Este artigo apresenta o Tigrinho Simulator, um jogo 2D desenvolvido em Python com a biblioteca Pygame que busca simular a vida de um viciado em bets, com três minigames de aposta (roleta europeia, blackjack e caça-níquel). O objetivo do jogo é criticar as apostas e mostrar os danos que elas causam: o jogador interpreta um pai endividado que só vence se passar sete dias consecutivos sem apostar. O projeto foi construído com foco na qualidade da modelagem e na aplicação de conceitos de Programação Orientada a Objetos (POO), como abstração, classes abstratas, herança, polimorfismo, encapsulamento com *getters* e *setters*, composição e agregação. O sistema separa as regras dos jogos (pacote `modelos`) da interface (pacote `states`), usa o padrão *State* para gerenciar as telas e persiste o histórico de partidas em um banco de dados JSON.**

***Palavras-chave*—Programação Orientada a Objetos, Python, Pygame, jogos, apostas, padrão State, UML.**

I. Introdução

As apostas em plataformas digitais, popularmente conhecidas no Brasil como "jogo do tigrinho", tornaram-se um problema social relevante. O transtorno do jogo é reconhecido como um vício comportamental em manuais diagnósticos, e os jogos de azar são projetados matematicamente para que, no longo prazo, o apostador perca mais do que ganha. Esse fato é pouco intuitivo para quem joga, pois o resultado de cada rodada isolada parece depender de sorte. [5]

Este trabalho apresenta o *Tigrinho Simulator*, um jogo cujo objetivo declarado é "criticar as apostas e denunciar o dano que as mesmas podem causar". O jogador controla um pai endividado com um agiota, que mora com esposa e filha e possui um celular com aplicativos de apostas. As conversas com os familiares, as cutscenes de derrota e a limitação diária de apostas reforçam a mensagem de que apostar não resolve os problemas financeiros.

Do ponto de vista da disciplina, o foco do trabalho não é a complexidade do sistema, mas a qualidade da modelagem, a organização do código e a aplicação correta dos conceitos de POO. Por isso, as regras dos jogos foram isoladas em classes independentes da interface gráfica, o que facilita o entendimento e a validação. O código foi escrito em Python, com cerca de 2,9 mil linhas distribuídas em 26 módulos, e utiliza apenas a biblioteca externa Pygame [1].

O restante do artigo está organizado assim: a Seção II descreve o jogo; a Seção III apresenta as ferramentas e a organização do projeto; a Seção IV apresenta o diagrama UML; a Seção V detalha os conceitos de POO aplicados; a Seção VI descreve o banco de dados; a Seção VII trata da validação de entradas; a Seção VIII apresenta resultados e a Seção X conclui o trabalho. O código completo está disponível em um repositório do GitHub. [6]

II. Descrição do Jogo

A. Narrativa e objetivo

Após uma cutscene em que o agiota leva o carro da família como garantia, o jogador passa a controlar o pai, que se move lateralmente entre a casa e a rua. Dentro de casa é possível conversar com a esposa e a filha (tecla `E`), cujos diálogos, sorteados de uma lista, mostram o impacto emocional das dívidas, e abrir o celular (tecla `C`).

B. Mecânicas

O celular funciona como um menu de aplicativos: *Jogos* (Roleta, Blackjack e Tigrinho), *Runs* (histórico de partidas) e *iFood*. O jogador começa cada partida com R$ 5.000 e pode fazer, no máximo, 10 apostas por dia. Ao atingir o limite, o dia termina e a tela de noite é exibida. O aplicativo iFood representa a alternativa honesta: ele rende R$ 500 e encerra o dia sem nenhuma aposta.

As condições de término são:

- **Derrota:** o saldo acaba. Uma cutscene mostra a esposa abandonando a casa e, em seguida, a tela de *Game Over* permite recomeçar.
- **Vitória:** o jogador acumula 7 dias consecutivos sem apostar. A cutscene final mostra a família reunida ("Você venceu o vício").

A Fig. 1 mostra o fluxo entre as telas.

C. Os minigames

Roleta europeia. Possui 37 casas (0 a 36). O jogador pode apostar em uma cor (vermelho ou preto, pagamento 1:1), em par ou ímpar (1:1) ou em um número (35:1). O zero é verde e faz perder as apostas em cor e paridade. A roda é animada

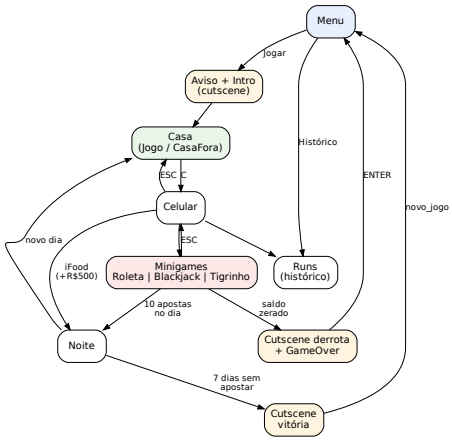


Figura 1: Fluxo de navegação entre as telas do jogo.

com desaceleração cúbica (*ease-out*), e o resultado é sorteado antes da animação, que apenas o revela.

Blackjack. Usa um sapato com quatro baralhos. O jogador pode pedir, parar ou dobrar; o *dealer* compra até atingir 17. O blackjack natural paga 3:2 e o empate devolve a aposta. O *dealer* joga carta a carta, com intervalo de 1 s, para a interface animar cada passo.

Caça-níquel (Tigrinho). Tem três rolos, uma fita circular de 20 símbolos e cinco linhas de pagamento (três horizontais e duas diagonais). O símbolo "tigre" é coringa. Cada giro custa o valor da aposta multiplicado por 5 linhas.

III. Materiais e Métodos

A. Ferramentas

O projeto foi desenvolvido em Python, com a biblioteca Pygame [1] para janela, entrada de teclado, imagens e desenho. O armazenamento usa o módulo `json` da biblioteca padrão [3], e os caminhos de arquivos usam `pathlib`. O empacotamento em executável é descrito no arquivo `TigrinhoRPG.spec` (PyInstaller) [2], e o módulo `config.py` detecta se o programa está congelado para localizar os recursos corretamente. Os diagramas UML deste artigo foram feitos com o Graphviz.

B. Organização em pastas

O projeto foi organizado por responsabilidade:

```
assets/        imagens e spritesheets
docs/          documentacao
src/
  core/        StateManager, Tracker
  modelos/     regras (roleta, blackjack,
               slots, carteira) e entidades
  states/      telas (uma classe por tela)
  utils/       HUD
  config.py    constantes e caminhos
  game.py      classe Game (loop principal)
  main.py      ponto de entrada
```

A decisão de projeto mais importante foi manter os módulos de `modelos` independentes do Pygame nas regras de jogo. Por exemplo, `roleta.py`, `blackjack.py` e `slots.py` não importam o Pygame, de modo que as regras podem ser executadas e testadas sem abrir uma janela. As classes em `states` apenas desenham, leem o teclado e chamam as regras.

IV. Modelagem UML

As Figuras 2, 3 e 4 apresentam o diagrama de classes completo do sistema, dividido em três partes para melhor legibilidade. Os atributos privados (−) e protegidos (#) seguem a convenção Python de prefixo _. Os membros marcados com ⋆ são abstratos, e `{get/set}` indica propriedade com *getter* e *setter*. Losango preenchido indica composição, losango vazado indica agregação e seta com triângulo vazado indica herança.

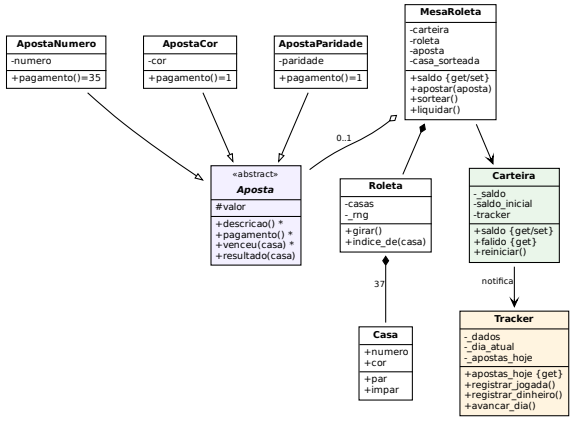


Figura 2: Diagrama UML (Parte 1): hierarquia de apostas, mesa da roleta, carteira e rastreador.

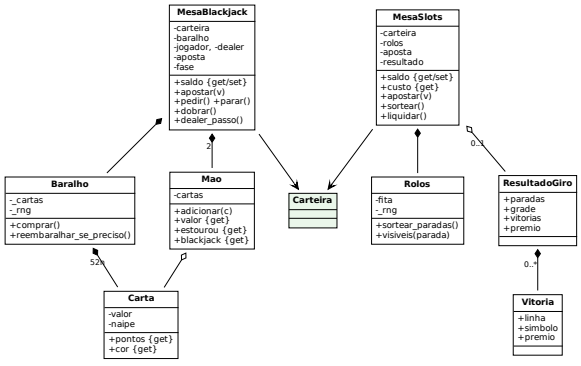


Figura 3: Diagrama UML (Parte 2): Blackjack e caça-níquel. As três mesas compartilham a mesma Carteira.

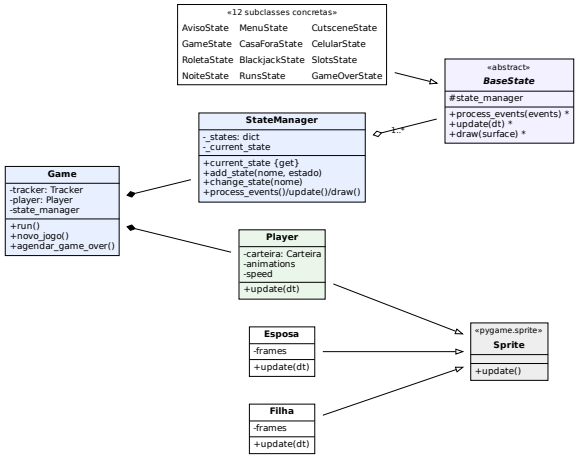


Figura 4: Diagrama UML (Parte 3): gerenciamento de telas (padrão *State*), classe Game e entidades animadas.

V. Conceitos de POO Aplicados

A. Abstração e classes abstratas

O projeto possui duas classes abstratas, ambas criadas com o módulo `abc` [4]. A primeira, `Aposta`, define o contrato de uma aposta da roleta com as propriedades abstratas `descricao` e `pagamento` e o método abstrato `venceu(casa)`. Já o método concreto `resultado(casa)` é um *template method*: usa os métodos abstratos para devolver o ganho líquido (`valor` × `pagamento` se venceu, ou `-valor` se perdeu), sem conhecer o tipo de aposta. A segunda, `BaseState`, declara os métodos `process_events`, `update` e `draw`, que toda tela deve implementar. Como usam `ABC`, nenhuma das duas pode ser instanciada diretamente.

B. Herança

Há três hierarquias: (i) `ApostaNumero`, `ApostaCor` e `ApostaParidade` herdam de `Aposta` e chamam `super().__init__`; (ii) as 12 telas herdam de `BaseState`; (iii) `Player`, `Esposa` e `Filha` herdam de `pygame.sprite.Sprite`, aproveitando a infraestrutura de grupos e desenho da biblioteca.

C. Polimorfismo

O polimorfismo aparece em dois pontos centrais. Primeiro, `MesaRoleta.liquidar()` chama `self.aposta.resultado(casa)` sem saber qual é a subclasse: um `ApostaNumero` paga 35:1, e os demais pagam 1:1, cada um com sua própria regra `venceu`. Segundo, o `StateManager` guarda um dicionário de telas e chama `process_events`, `update` e `draw` no estado ativo, sem condicionais por tipo de tela. Adicionar uma nova aposta ou tela exige criar uma subclasse, sem alterar o código existente (princípio aberto/fechado). Além disso, `Player`, `Esposa` e `Filha` redefinem `update(dt)` com comportamentos diferentes.

D. Encapsulamento, getters e setters

Os atributos internos usam prefixo _ e o acesso é feito por `@property`. A classe `Carteira` é o exemplo principal: o saldo fica em `_saldo`, o *getter* o devolve e o *setter* calcula a diferença e notifica o `Tracker`, que registra ganhos e perdas. Assim, qualquer alteração de saldo no jogo é contabilizada em um único ponto. A propriedade `falido` é somente leitura. O `Tracker` expõe `dia_atual`, `apostas_hoje`, `dias_sem_apostar` e `total_runs` apenas como leitura, e `obter_run()` devolve uma cópia (`copy.deepcopy`) para que a interface não altere o histórico por engano. O `StateManager` expõe `current_state` somente para leitura. As classes de domínio também usam propriedades derivadas, como `Carta.pontos`, `Mao.valor`, `Mao.estourou` e `Casa.par`.

E. Composição, agregação e delegação

A classe `Game` compõe o `StateManager`, o `Tracker` e o `Player`. O `Player` compõe uma `Carteira`. Cada mesa compõe seus objetos de regra: `MesaRoleta` tem uma `Roleta` (com 37 `Casa`), `MesaBlackjack` tem um `Baralho` e duas `Mao`, e `MesaSlots` tem `Rolos`. Já a `Carteira` é recebida por agregação: as três mesas recebem a mesma instância no construtor, então o saldo é único para todos os jogos. As propriedades `saldo` das mesas delegam para a carteira.

F. Injeção de dependência

Os construtores recebem as dependências como parâmetros opcionais: `Roleta(rng=None)`, `Baralho(rng, cartas)`, `Rolos(rng, fita)` e `Carteira(saldo, tracker)`. Isso permite substituir o gerador aleatório por uma semente fixa ou fixar a ordem das cartas, tornando os resultados reproduzíveis. A interface também oferece um modo de teste (tecla `T`) que força o resultado do sorteio.

G. Padrão State e acoplamento

O ciclo de vida das telas segue o padrão *State*. A classe `Game` registra as telas no `StateManager` por nome, e cada tela troca de estado chamando `change_state("Nome")`. O gerenciador ignora trocas em menos de 250 ms, o que evita que um mesmo toque de tecla dispare duas transições. A classe `CutsceneState` é reutilizada três vezes, com listas de cenas diferentes (abertura, derrota e vitória) passadas por parâmetro.

H. Resumo dos requisitos

A Tabela I relaciona cada requisito da disciplina ao ponto do código que o atende.

Tabela I: Requisitos de POO e onde são atendidos

Requisito	Implementação
Classe abstrata	`Aposta`, `BaseState` (`ABC`)
Herança	`Aposta`→3 tipos; `BaseState`→12 telas; `Sprite`→`Player`, `Esposa`, `Filha`
Polimorfismo	`aposta.resultado()`; `estado.draw()`; `update()` dos sprites
Encapsulamento	Atributos com _; acesso por `@property`
Getter/setter	`Carteira.saldo` e `saldo` das mesas
Banco de dados	`Tracker` persiste em `runs_data.json`
Erros de entrada	`ValueError` nas regras, tratado nas telas
≥3 classes	36 classes de domínio e de interface
UML	Figs. 2–4

VI. Banco de Dados

O histórico de partidas (*runs*) é persistido em um arquivo JSON pela classe `Tracker`, que funciona como camada de acesso a dados. Cada partida iniciada cria um registro com a estrutura abaixo:

```
{"runs": [
  {"numero": 1,
   "ganhos": 1250, "perdas": 4300,
   "dias_jogados": 3,
   "jogos": {"Roleta": 12,
             "Blackjack": 5,
             "Tigrinho": 9}}
]}
```

As operações do CRUD cobertas são: *criar* (`adicionar_run`), *ler* (`_carregar`, `obter_run`, `total_runs`) e *atualizar* (`registrar_dinheiro`, `registrar_jogada`, `avancar_dia`). Toda atualização chama `_salvar`, de modo que o arquivo nunca fica defasado em relação à memória. A leitura é tolerante a falhas: se o arquivo não existe, está corrompido ou não tem a chave `runs` no formato esperado, o `Tracker` inicia com um histórico vazio. O caminho do arquivo é definido em `config.py` e fica ao lado do executável quando o jogo está empacotado.

O mesmo objeto guarda o estado temporário da partida (dia atual, apostas do dia e dias consecutivos sem apostar). A tela *Runs* lê esses dados e permite navegar entre as partidas com as setas.

VII. Tratamento de Erros

A validação segue uma divisão de responsabilidades. As classes de regra lançam `ValueError` com mensagens claras quando uma operação é inválida: aposta com valor não positivo ou maior que o saldo, aposta em andamento, número fora de 0–36, cor ou paridade desconhecida, carta com valor ou naipe inválido, jogada fora de turno e baralho vazio. As telas capturam a exceção em blocos `try/except ValueError` e exibem a mensagem ao jogador, sem encerrar o programa. Como a entrada é feita por teclas, o valor da aposta é limitado por faixa (mínimo e saldo máximo) e alterado em passos fixos, o que reduz o espaço de erros. Os recursos gráficos ausentes (por exemplo, imagens de cutscene ou de símbolos) são tratados com `FileNotFoundError` e `pygame.error`, com *fallback* para texto colorido. O arquivo de histórico também é protegido, como descrito na seção anterior.

VIII. Resultados e Validação

A. Funcionamento do programa

O programa busca reproduzir a vida de um pai de família viciado em apostas, com o objetivo de demonstrar os estragos que esse vício é capaz de causar na vida de uma pessoa. Para cumprir com o objetivo proposto, foram desenvolvidas mecânicas para cutscenes Fig. 5 e diálogos com a família Fig. 6 , com o intuito de trazer peso a história e as ações do jogador. Além disso, o jogo disponibiliza três dos principais jogos de azar praticados no mundo todo, Figuras 7, 8, 10. Por mais que seja possível ganhar, se o jogador continuar apostando, assim como na vida real, a chance de prejuízo é altissíma, e aumenta a cada nova aposta.Existem dois finais para o jogo. O primeiro, e o único jeito de terminar o jogo vitorioso, é ficando 7 dias sem jogar, simbolizando o fim do vício, e o ínicio de uma mudança. Em contraponto, se o saldo do jogador chegar a 0 em qualquer momento, o jogo termina, com o final ruim. Também é possível acessar através do menu, ou do celular Fig. 11, o histórico de runs 12, que muitas vezes se torna mais uma forma de relembrar ao jogador que as perdas geralmente sobressaem em muito os ganhos.
As capturas de tela a seguir demonstram algumas das principais mecânicas implementadas no jogo, em funcionamento.



Figura 5: Cena inicial do jogo - Cutscene do agiota.

B. Verificação funcional

A validação foi feita por execução e jogo manual das telas, cobrindo: transição entre todos os estados da Fig. 1; limite de 10 apostas por dia; fim de dia e contagem de dias sem apostar; condição de vitória em 7 dias; derrota com saldo zerado; navegação do histórico; e entrada inválida (aposta maior que o saldo). O modo de teste do jogo (tecla `T`) força o resultado de roleta e caça-níquel, e a tecla `J` do caça-níquel força um *jackpot*. Como as regras estão separadas da interface e aceitam semente aleatória, elas estão prontas para receber testes automatizados, o que não foi implementado nesta entrega.

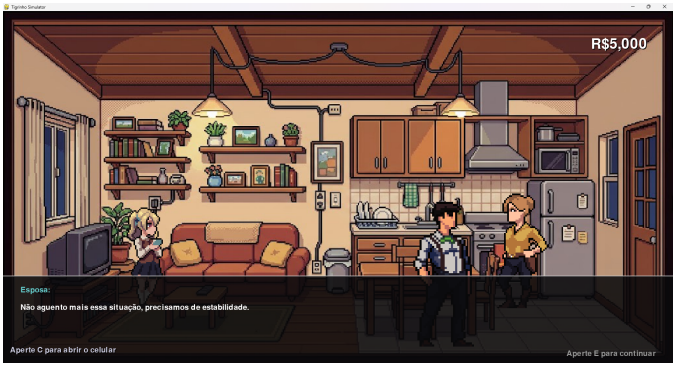


Figura 6: Dialogo com a esposa, dentro da casa.



Figura 7: Tela de jogabilidade da roleta.



Figura 8: Tela de jogabilidade do caça-níquel.

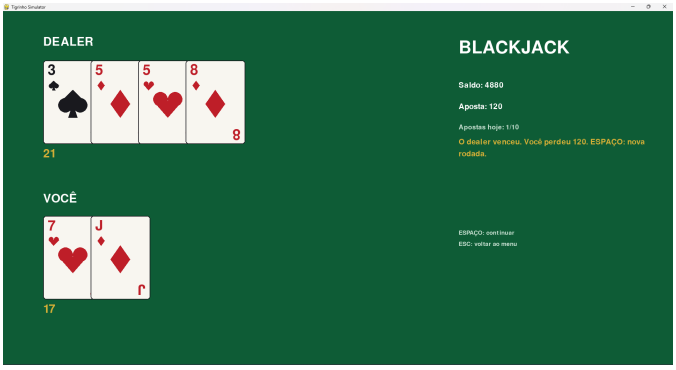


Figura 9: Tela de jogabilidade do BlackJack.

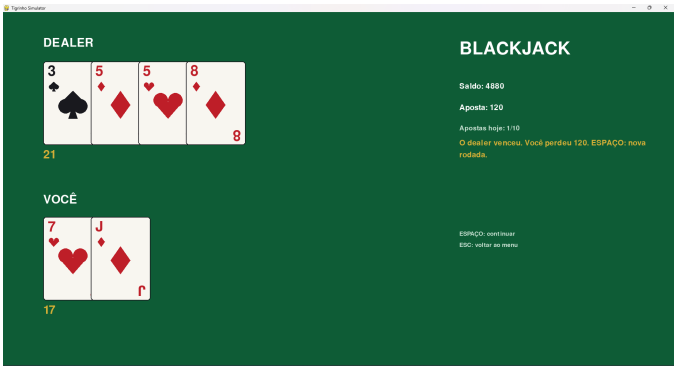


Figura 10: Tela de jogabilidade do BlackJack.



Figura 11: Interface do celular.

IX. Limitações e Trabalhos Futuros

O banco de dados é um arquivo JSON, adequado ao volume pequeno de dados, mas não oferece consultas, transações ou acesso concorrente; uma migração para SQLite seria natural. Outras melhorias são: testes automatizados com `unittest` usando as sementes injetáveis; saves de partida; mais mini-games reutilizando a hierarquia de apostas. O menu também pode ser unificado para que os atalhos numéricos coincidam com a ordem das opções.

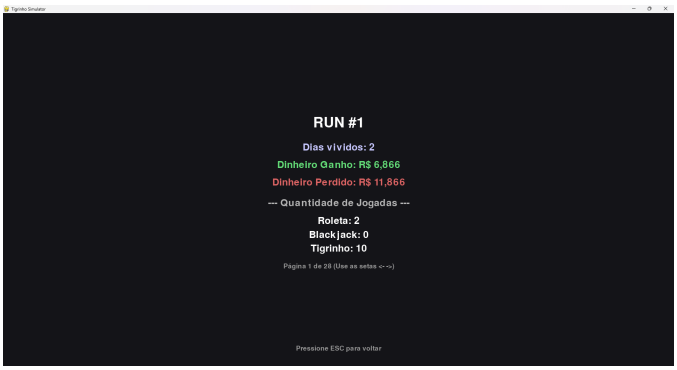


Figura 12: Histórico de jogadas.

X. Conclusão

O Tigrinho Simulator atende aos requisitos de POO propostos: possui mais de três classes, herança, polimorfismo, classes abstratas, encapsulamento com *getters* e *setters*, tratamento de erros de entrada, persistência em banco de dados e diagrama UML. A separação entre regras (`modelos`) e interface (`states`), junto com o padrão *State*, deixou o código organizado e extensível: novos tipos de aposta e novas telas são adicionados por meio de novas subclasses. O jogo também cumpre a função de crítica social: com uma taxa de retorno naturalmente desfavorável ao jogador; com limite diário de apostas e uma vitória que só vem de parar de apostar, a mecânica mostra ao jogador que a aposta é um mau negócio.

Referências

[1] Pygame Community, "Pygame documentation." Disponível em: https://www.pygame.org/docs/.

[2] PyInstaller Development Team, "PyInstaller Manual." Disponível em: https://pyinstaller.org/en/stable/.

[3] Python Software Foundation, "json — JSON encoder and decoder." Disponível em: https://docs.python.org/3/library/json.html.

[4] Python Software Foundation, "abc — Abstract Base Classes." Disponível em: https://docs.python.org/3/library/abc.html.

[5] Conselho Federal de Psicologia. A Psicologia frente ao mundo digital: orientações para a atuação profissional com crianças e adolescentes. 1. ed. Brasília, DF: Conselho Federal de Psicologia, 2025.

[6] SANTOS, Davi Menezes dos; PINHEIRO, Renato Alexandre Moura; MELLO, Yan Martelleto de; VERÇOSA, Arthur Lobo Nunes. Tigrinho RPG. Repositório da disciplina de Programação Orientada à Objetos. GitHub, 2026. Disponível em: https://github.com/davimenezes470-pixel/TP-POO-2026-2. Acesso em: 10 nov. 2026.